Stress + Chaos — Phase 1 Evidence (§11.4.85 Lava equivalent)

Date: 2026-05-31 (68th-cycle follow-up). **Scope:** lava-api-go in-process stress + chaos suite (`tests/stress/`). **Result:** the suite was **authored, built, and ACTUALLY RAN**; verdict **PASS**, `go test` **EXIT=0**.

Verbatim run output (`/tmp/stress_run.log`)

```
=== RUN   TestStressChaos
    api_stress_test.go:252: evidence written: evidence/
2026-05-31T07-58-03Z/stress-chaos.json (verdict=PASS)
--- PASS: TestStressChaos (0.81s)
PASS
ok      digital.vasic.lava.apigo/tests/stress   1.625s
EXIT=0
```

Command invoked (from repo root, this session):

```
chmod +x scripts/run-chaos-stress.sh
cd lava-api-go
GOMAXPROCS=2 go test -tags stress -run TestStressChaos -v ./tests/
        stress/...   # → EXIT=0, PASS
```

Provenance from the evidence file: git SHA `a6c278d076748ead360b46bc360b39c412260dd1`, `go1.26.2 darwin/arm64` on host `Mistborn.local`, started `2026-05-31T07:58:02Z`, wall 0.80s. Leak guards: goroutines 3 → 5 (clean — httptest server workers), open FDs 0 → 0 (`/dev/fd` read returned 0 on this host, recorded as 0 not faked).

Captured per-dimension metrics (verbatim from `evidence/2026-05-31T07-58-03Z/stress-chaos.json`)

Dim	Name	Ran	Status	Reqs	2xx	4xx	5xx	errR
S1	sustained-load	true	PASS	500	500	0	0	0.000
S2	concurrent-contention	true	PASS	64	64	0	0	0.000
C1	fault-injection-recovery	true	PASS	300	200	0	100	0.333
C2	latency-injection	true	PASS	100	0	0	0	0.000
C5	malformed-input	true	PASS	100	0	100	0	0.000
C4a	rate-limiter-trip	true	PASS	30	10	20	0	0.000
C3	dependency-kill-postgres	**false**	**OPERATOR_GATED**	0	0	0	0	—
C4b	pool-exhaustion	**false**	**OPERATOR_GATED**	0	0	0	0	—

Chaos recovery observations (verbatim)

- **C1** (dependency-unavailable-503): errRate **during** fault = **1.000**, **after** fault = **0.000**, recovery in **1 request**. Fault returns a clean 503 (no panic / no hang); recovers to 200 the very first request after the fault clears. This is the load-bearing chaos proof — a real dependency outage produces a clean error, not a crash, and recovery is immediate.
- **C2** (5ms-per-request latency injection): errRate 0.000; p50 degraded to **6.04ms** under the 5ms injection (server stays responsive, percentiles degrade gracefully, no hang).
- **C4a** (rate-limit exhaustion): first **10 → 200**, remaining **20 → 429**, deterministic; 0 × 5xx.
- **C5** (oversized 1KB path segment under load): **100/100 → clean 400**, 0 × 5xx (no panic).

What ran vs what is operator-gated

All six in-process dimensions (S1, S2, C1, C2, C4a, C5) **ran and passed** on this macOS host with no Postgres, no emulator, no sudo, in 0.80s wall-clock.

C3 (Postgres-kill) and C4b (pgx-pool-exhaustion) are **OPERATOR_GATED by design** — they need a real Postgres container under podman. The evidence file records them `"ran": false, "status": "OPERATOR_GATED"` with **no fabricated metrics** (§6.J / §11.4.6). They run via `scripts/run-chaos-stress.sh --with-podman` once the Phase 1.b real-Postgres chaos driver lands (see DESIGN.md §6). This gating is a deliberate phase boundary, not a relay/tooling failure.

Falsifiability (§11.4.85 / Sixth Law clause 2)

The gating thresholds are conservative and structural, so the PASS is meaningful:

- S1/S2 assert errRate==0 against a trivial `/health` — only a broken Go HTTP stack could flake them.
- C1 asserts ≥0.99 error-rate *during* the injected fault AND 0.0 *after* + a 2xx returns — flip the fault-injection middleware off and the "during fault" assertion (`durErr < 0.99`) fires; leave it stuck on and the post-clear `postErr > 0 || post2 == 0` assertion fires. Either break is caught.
- C4a/C5 assert exact status-code partitions — change the rate-limit cap or the oversize threshold and the counts diverge from the expected `(10,20)` / `(100 × 4xx)` and the test FAILs.

Absolute latency magnitude is **recorded, never asserted on**, precisely so the evidence is descriptive without being host-jitter-flaky.